

SGA Database System

Spring Boot CRUD Application — Authors & Books

Project	SGA Library Management
Stack	Spring Boot 3.2.5, Spring Data JPA, JSP, JSTL, H2
Language	Java 17
Build Tool	Maven
Tests	JUnit 5 + Mockito (16 tests, all passing)
Author	_____
Date	_____
GitHub URL	_____

1. Project Overview

This project is a Spring Boot web application that manages information for two related entities: **Authors** and **Books**. It implements three CRUD operations — **Create**, **Read**, and **Update** — through a layered architecture (entity, repository, service, controller, view) and exposes the functionality via JSP pages styled with custom CSS. The data layer uses an H2 in-memory database, and the application is seeded with 10 authors and 10 books on startup. The repository layer additionally exposes a custom JPQL **INNER JOIN** query that returns books together with their authors as a flat projection.

Unit tests cover the repository layer (using `@DataJpaTest` against an embedded H2) and the service layer (using Mockito), including data-integrity-violation scenarios.

2. Entity Relationship Design

The two chosen entities are related as **One Author has Many Books**, and **each Book belongs to exactly one Author**. This is modelled as a bidirectional JPA association: `@OneToMany` on the Author side and `@ManyToOne` on the Book side, with the Book table owning the foreign key column `author_id`.

Diagram

Author		1	N	Book	
id	BIGINT (PK)			id	BIGINT (PK)
name	VARCHAR			author_id	BIGINT (FK)
email	VARCHAR (U)			title	VARCHAR
nationality	VARCHAR			isbn	VARCHAR (U)
				genre	VARCHAR
				price	DOUBLE

(U) = UNIQUE constraint (FK) = foreign key on author_id

Constraints & Validation

- Author.email is **unique** and not null.
- Book.isbn is **unique** and not null.
- Book.author_id is a non-null foreign key — every book must reference an author.
- Bean Validation annotations (`@NotBlank`, `@Positive`, `@Size`) enforce field-level rules.
- Cascade *ALL* with orphan removal on the Author→Books side keeps the children consistent with their parent.

Author entity (excerpt)

```
@Entity
@Table(name = "authors",
      uniqueConstraints = @UniqueConstraint(columnNames = "email"))
public class Author {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank @Size(max = 100)
    @Column(nullable = false, length = 100)
```

```
private String name;

@NotBlank
@Column(nullable = false, unique = true, length = 150)
private String email;

@Column(length = 60)
private String nationality;

@OneToMany(mappedBy = "author", cascade = CascadeType.ALL,
            orphanRemoval = true, fetch = FetchType.LAZY)
private List<Book> books = new ArrayList<>();
}
```

Book entity (excerpt)

```
@Entity
@Table(name = "books",
       uniqueConstraints = @UniqueConstraint(columnNames = "isbn"))
public class Book {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank @Column(nullable = false, length = 200)
    private String title;

    @NotBlank @Column(nullable = false, unique = true, length = 20)
    private String isbn;

    @Column(length = 60) private String genre;

    @NotNull @Positive @Column(nullable = false)
    private Double price;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "author_id", nullable = false)
    private Author author;
}
```

3. Database Population (Sample Data)

On application startup, a Spring *CommandLineRunner* bean (**DataInitializer**) inserts 10 authors and 10 books if the tables are empty. This seeds the DB so the JSP listings have content immediately.

```
@Bean
CommandLineRunner seedDatabase(AuthorRepository authorRepo,
                               BookRepository bookRepo) {
    return args -> {
        if (authorRepo.count() > 0) return;
        List<Author> authors = List.of(
            new Author("George Orwell", "...", "British"),
            new Author("Jane Austen", "...", "British"),
            /* ... 10 authors total ... */);
        authorRepo.saveAll(authors);
        List<Book> books = List.of(
            new Book("1984", "978-0451524935",
                    "Dystopian", 14.99, authors.get(0)),
            /* ... 10 books total, each linked to an author ... */);
        bookRepo.saveAll(books);
    };
}
```

[Insert screenshot here]

Figure 1 — H2 console showing populated AUTHORS and BOOKS tables

Tip: open <http://localhost:8080/h2-console> with JDBC URL `jdbc:h2:mem:librarydb` to take this screenshot.

4. Create Operation

The Create flow is implemented as an HTML form rendered by a JSP, submitted to a controller method that delegates to the service layer, which in turn persists the entity through the repository. Bean Validation annotations are evaluated on submission, and *DataIntegrityViolationException* is caught explicitly so that duplicate-email or duplicate-ISBN attempts surface a friendly inline message rather than a 500 page.

Controller

```
@PostMapping
public String create(@Valid @ModelAttribute("book") Book book,
                    BindingResult result,
                    @RequestParam("authorId") Long authorId,
                    Model model,
                    RedirectAttributes redirectAttrs) {
    if (result.hasErrors()) {
        model.addAttribute("authors", authorService.findAll());
        return "books/form";
    }
    try {
        bookService.create(book, authorId);
        redirectAttrs.addFlashAttribute(
            "message", "Book created successfully.");
        return "redirect:/books";
    } catch (DataIntegrityViolationException ex) {
        model.addAttribute("authors", authorService.findAll());
        model.addAttribute("error", ex.getMessage());
        return "books/form";
    }
}
```

Service (integrity-violation translation)

```
public Book create(Book book, Long authorId) {
    Author author = authorService.findById(authorId);
    book.setAuthor(author);
    try {
        return bookRepository.save(book);
    } catch (DataIntegrityViolationException ex) {
        throw new DataIntegrityViolationException(
            "Could not save book: a book with ISBN '"
            + book.getIsbn() + "' already exists.", ex);
    }
}
```

JSP form (excerpt)

```
<form:form modelAttribute="book" method="post" action="/books">
  <div class="field">
    <label>Title</label>
    <form:input path="title"/>
    <form:errors path="title" cssClass="field-error"/>
  </div>
  <!-- isbn, genre, price, author fields ... -->
  <button type="submit" class="btn">Create Book</button>
</form:form>
```

[Insert screenshot here]

Figure 2 — Add New Book form (/books/new)

[Insert screenshot here]

Figure 3 — Add New Author form (/authors/new)

[Insert screenshot here]

Figure 4 — Inline error after submitting a duplicate ISBN (data-integrity violation)

5. Read Operation

Two list pages are provided. **/books** and **/authors** render plain entity lists from *findAll()*. A third page, **/books/with-authors**, renders the result of a custom JPQL inner-join query, projecting both Book and Author columns into a flat DTO (*BookAuthorView*). This satisfies the requirement to expose a custom repository query that joins the two entities.

Controller

```
@GetMapping
public String list(Model model) {
    model.addAttribute("books", bookService.findAll());
    return "books/list";
}

@GetMapping("/with-authors")
public String listWithAuthors(Model model) {
    model.addAttribute("rows", bookService.findAllBooksWithAuthors());
    return "books/with-authors";
}
```

JSP listing (binding via JSTL)

```
<table>
  <thead><tr><th>ID</th><th>Title</th><th>ISBN</th>
    <th>Author</th><th>Actions</th></tr></thead>
  <tbody>
    <c:forEach var="b" items="${books}">
      <tr>
        <td>${b.id}</td>
        <td><c:out value="${b.title}"/></td>
        <td><c:out value="${b.isbn}"/></td>
        <td><c:out value="${b.author.name}"/></td>
        <td><a class="btn btn-small"
          href="/books/${b.id}/edit">Edit</a></td>
      </tr>
    </c:forEach>
  </tbody>
</table>
```

[Insert screenshot here]

Figure 5 — Author list page (/authors)

[Insert screenshot here]
Figure 6 — Book list page (/books)

6. Custom Inner-Join Query

The repository defines a JPQL query using **INNER JOIN** on the *book.author* association, projecting columns from both entities into a constructor-expression DTO. This means only one round-trip to the database, no lazy-loading per row, and the result is shaped exactly the way the JSP needs it.

Repository method

```
@Query("""
    SELECT new com.sga.library.dto.BookAuthorView(
        b.id, b.title, b.isbn, b.genre, b.price,
        a.id, a.name, a.email, a.nationality)
    FROM Book b
    INNER JOIN b.author a
    ORDER BY a.name ASC, b.title ASC
    """)
List<BookAuthorView> findAllBooksWithAuthors();
```

Generated SQL (from Hibernate)

```
SELECT b1_0.id, b1_0.title, b1_0.isbn, b1_0.genre, b1_0.price,
       a1_0.id, a1_0.name, a1_0.email, a1_0.nationality
FROM   books b1_0
JOIN   authors a1_0 ON a1_0.id = b1_0.author_id
ORDER BY a1_0.name, b1_0.title;
```

BookAuthorView DTO

```
public class BookAuthorView {
    private final Long bookId;
    private final String title, isbn, genre;
    private final Double price;
    private final Long authorId;
    private final String authorName, authorEmail, authorNationality;

    // constructor matching the SELECT new ... in JPQL
    // + getters
}
```

[Insert screenshot here]

Figure 7 — Books + Authors inner-join page (/books/with-authors)

7. Update Operation

Each list page exposes an Edit button that loads a pre-populated form. The same controller pattern (POST + redirect-after-submit) is reused. The service fetches the persisted entity, applies the form fields, and saves — keeping the JPA identity stable. Integrity violations (e.g. assigning an ISBN that already belongs to another book) are caught and re-rendered as inline form errors.

Controller

```
@PostMapping("/{id}")
public String update(@PathVariable Long id,
                    @Valid @ModelAttribute("book") Book book,
                    BindingResult result,
                    @RequestParam("authorId") Long authorId,
                    Model model,
                    RedirectAttributes redirectAttrs) {
    if (result.hasErrors()) {
        model.addAttribute("authors", authorService.findAll());
        return "books/edit";
    }
    try {
        bookService.update(id, book, authorId);
        redirectAttrs.addFlashAttribute("message",
            "Book updated successfully.");
        return "redirect:/books";
    } catch (DataIntegrityViolationException ex) {
        model.addAttribute("authors", authorService.findAll());
        model.addAttribute("error", ex.getMessage());
        return "books/edit";
    }
}
```

Service

```
public Book update(Long id, Book updated, Long authorId) {
    Book existing = findById(id);
    existing.setTitle(updated.getTitle());
    existing.setIsbn(updated.getIsbn());
    existing.setGenre(updated.getGenre());
    existing.setPrice(updated.getPrice());
    if (authorId != null) {
        existing.setAuthor(authorService.findById(authorId));
    }
    return bookRepository.save(existing);
}
```

[Insert screenshot here]

Figure 8 — Edit Book form pre-populated with existing data

[Insert screenshot here]

Figure 9 — Book list reflecting the updated record

8. Testing & Validation

The project ships with 16 unit tests across four classes. Repository tests use `@DataJpaTest` with Spring's auto-configured embedded H2 to verify the custom inner-join query and unique-constraint enforcement. Service tests use Mockito to isolate the business logic from the DB, including exception-translation paths.

Inner-join repository test

```
@DataJpaTest
class BookRepositoryTest {
    @Test
    void findAllBooksWithAuthors_returnsInnerJoinedRows() {
        Author a = authorRepository.save(
            new Author("Alpha Author", "a@x.com", "USA"));
        bookRepository.save(
            new Book("Alpha Book", "A-001", "Fiction", 10.0, a));

        List<BookAuthorView> rows =
            bookRepository.findAllBooksWithAuthors();

        assertThat(rows).hasSize(1);
        assertThat(rows.get(0).getAuthorName())
            .isEqualTo("Alpha Author");
    }
}
```

Service-level integrity-violation test

```
@Test
void create_translatesIntegrityViolation() {
    when(authorService.findById(10L)).thenReturn(author);
    when(bookRepository.save(any(Book.class)))
        .thenThrow(new DataIntegrityViolationException("dup"));

    assertThatThrownBy(() -> bookService.create(input, 10L))
        .isInstanceOf(DataIntegrityViolationException.class)
        .hasMessageContaining("ISBN-DUP");
}
```

[Insert screenshot here]

Figure 10 — Maven test output: 16 tests passed

9. Challenges Faced

- **JSP packaging with embedded Tomcat.** Spring Boot's default jar packaging does not serve JSPs from the classpath. Resolution: keep JSPs under `src/main/webapp/WEB-INF/views`, declare `tomcat-embed-jasper` with scope `provided`, and wire the view resolver via `spring.mvc.view.prefix / suffix` in `application.properties`.
- **Mapping the inner-join into a flat row.** Returning `List<Object[]>` from the join query forced index-based access in the JSP, which is fragile. Resolution: use a JPQL constructor expression (`SELECT new com.sga.library.dto.BookAuthorView(...)`) so the result is a typed DTO with proper getters, which the JSP renders cleanly via EL.
- **Distinguishing data-integrity violations from generic errors.** Saving a duplicate email or ISBN raised a low-level Hibernate exception that was not user-friendly. Resolution: wrap each `save()` in the service layer with a try/catch on `DataIntegrityViolationException` and re-throw a contextual message, then have controllers render it as an inline form error rather than crashing into a 500 page.
- **Avoiding lazy-loading issues in JSPs.** Rendering a list of books inside JSP triggered `LazyInitializationException` when the rendering happened outside an active session. Resolution: rely on `spring.jpa.open-in-view` for the simple list pages, and use the inner-join projection for the join page so the JSP never needs to traverse a lazy association.
- **Two-way relationship with Jackson serialisation.** The bidirectional Author ↔ Books relationship would have caused infinite recursion if exposed to JSON. Resolution: annotate the `books` collection on Author with `@JsonIgnore` so JSON callers see a clean Author payload while JPA still walks the relationship internally.

10. Submission Details

GitHub URL: _____

How to run locally

```
git clone <github-url>
cd library
mvn spring-boot:run
# then open http://localhost:8080/
```

How to run tests

```
mvn test    # 16 tests, all passing
```

Useful URLs

- / — home
- /authors — list / create / edit authors
- /books — list / create / edit books
- /books/with-authors — custom inner-join report
- /h2-console — H2 web console (JDBC: jdbc:h2:mem:librarydb)